Worked Examples That Compute Themselves: Step-by-Step Explainers and Generated Interactive Labs for Lecture Decks

Swayam Devendra Panchal, Kawaljeet Singh Bharaj, Jeevesh Bharat Bodhani, Rishit Pankajkumar Lahoti, and Pratik Kanani

Department of Artificial Intelligence and Data Science,
Dwarkadas J. Sanghvi College of Engineering, Mumbai, India
panchalswayam35@gmail.com

Abstract. Lecture slides in technical courses often contain a solved problem—a Viterbi trellis, a CYK chart, a Bayes calculation— shown as a finished table. Students see the answer but not the order of decisions that produced it. We present two components of Learnova, a system that rebuilds existing lecture decks, which turn such slides into something a student can work through. The first is a set of exact solvers that recognise a problem in slide text and produce a beginner-level explanation, one step at a time, with every number computed symbolically rather than generated by a language model. The second is a lab generator that attaches an interactive step-through page to each topic of a deck: hand-built labs for common topics, and labs generated for other topics either by a language model, whose code is checked before use, or offline from the slide's own structure. On a held-out set of 26 problems written for this study the solvers detected all problems and gave the correct answer in every case after one parsing fix, with no false detections on 12 ordinary lecture sentences and a median time of 0.7 ms. On three real NLP decks the system attached 6, 11 and 14 labs without any API key.

Keywords: worked examples · step-by-step solutions · interactive simulations · NLP education · presentation generation

1 Introduction

Worked examples are one of the best-supported techniques in instructional research: novices learn more from studying a solved problem step by step than from solving the same problem unaided [12, 1, 10]. The benefit depends on two things that a lecture slide usually lacks. Each step must be visible on its own, and the reason for the step must be stated. A slide that shows a completed Viterbi trellis gives neither.

Our course decks contain many such slides. When we rebuilt them with Learnova we wanted every solved problem to become a stepper that a student can move through, and every hard topic to have a small laboratory where the student can change the input and watch the algorithm run. Two constraints

shaped the design. First, the numbers must be right. Language models make arithmetic errors and confident mistakes [5, 7], and an explanation that is wrong in one step does more harm than no explanation. Second, the system must work in a classroom without network access or API credit.

The contributions of this paper are: (i) a detector and a library of exact solvers that turn a problem in slide text into an explained, step-by-step solution, including NLP algorithms; (ii) a lab bench and a lab selection policy that attach one interactive page per topic; (iii) a lab generator with two paths—model-written code that is statically checked, and an offline builder that uses the slide's structure; and (iv) an evaluation on a held-out problem set and on real decks.

2 Related Work

Step-by-step solutions are offered by computer algebra services such as Wolfram|Alpha [15], whose explanations target the mathematics rather than a specific lecture. Intelligent tutoring systems provide step-level feedback and approach the effectiveness of human tutoring [13], but they require authoring for each domain. Algorithm visualisation has a long history in computing education; meta-analyses report that the learner's engagement with the visualisation, not the animation itself, predicts learning [4, 9]. PhET simulations show how inquiry-oriented interactive pages can be used at scale [14]. The ICAP framework [2] and the broader evidence for active learning [3] support moving students from passive viewing to interactive work, and retrieval practice [11] motivates the "predict before you reveal" prompts in our labs.

3 Computed Step-by-Step Explainers

3.1 Representation

An explainer is a tuple $E = (\kappa, p, \langle s_1, \ldots, s_n \rangle, a, \rho)$: the problem type κ, the problem statement p, a sequence of steps, the answer a and a short recap ρ. Each step $s_i = (\mathit{title}_i, \mathit{work}_i, \mathit{why}_i, \mathit{rule}_i, G_i)$ has a short action title, one or more lines of working (with the part that changed marked for highlighting), a plain-language reason, the general rule behind the step and, for table algorithms, the current state of the table G_i with the cells to highlight. An explainer is accepted only if $n \geq 3$ and every step has both a title and a reason.

3.2 Detection

Slide text rarely contains a clean problem statement. The detector therefore builds candidate strings in order: the whole text (for grammars and systems of equations), neighbouring pairs of lines that both contain "=", each line and each sentence. Each candidate is offered to the solvers in a fixed order and the first accepted explainer wins:

$$E^* = \text{first}\{\, S_j(c) \neq \varnothing : c \in \mathit{Cand}(x),\ S_j \in \mathcal{S} \,\}. \tag{1}$$

Table 1. Solvers and what each step shows

Solver	Steps
Linear equation	write, expand, collect unknowns, remove constant, divide, check by substitution
Quadratic	standard form, factor or formula, both roots, check
Pair of linear equations	elimination with the multiplier shown, back-substitution, check
Percent, interest, speed, mean, median, probability, Pythagoras	formula, substitution, result with units
Bayes' rule	name the quantities, numerator, denominator, posterior
Derivative, integral	rule applied term by term
CYK parsing	chart cell by cell, with the rules that fire
Bigram probability	counts, each conditional probability, product
Naive Bayes (WSD)	priors, smoothed likelihoods, scores, decision
Viterbi decoding	initialisation, recursion per time step, back-pointers, path

For algebra, the equation is extracted from the sentence by removing leading words ("Solve", "Find x if", "Solve for y given") and trailing clauses, and is rejected if either side still contains a word of two or more letters. This conservative rule is what keeps ordinary lecture sentences from being treated as problems.

3.3 Solvers

All arithmetic uses SymPy [8] or exact fractions. Table 1 lists the solvers. We describe the NLP solvers, which are the main reason for this work.

Bigram model. For a corpus with sentence markers, the maximum-likelihood estimate [6]

$$P(w_i \mid w_{i-1}) = \frac{C(w_{i-1} w_i)}{C(w_{i-1})} \tag{2}$$

is computed for each pair in the query sentence, and the sentence probability is the product $P(w_{1:n}) = \prod_i P(w_i \mid w_{i-1})$, shown as an exact fraction and as a decimal.

Naive Bayes word sense disambiguation. For senses s with training contexts, vocabulary size V, and N_s the number of word tokens labelled s:

$$\hat{s} = \arg\max_s \; P(s) \prod_j P(f_j \mid s), \qquad P(f_j \mid s) = \frac{C(f_j, s) + 1}{N_s + V}, \tag{3}$$

where $P(s)$ is the fraction of training contexts with sense s. Words of the test context that never occur in training are listed and skipped, which is the convention used in our course.

Viterbi decoding. For an HMM with start probabilities π_j, transitions a_{ij} and emissions $b_j(o)$, the solver fills

$$v_1(j) = \pi_j \, b_j(o_1), \qquad v_t(j) = \max_i \, v_{t-1}(i) \, a_{ij} \, b_j(o_t), \qquad \psi_t(j) = \arg\max_i \, v_{t-1}(i) \, a_{ij}, \tag{4}$$

showing every candidate product for every cell, and then follows the back-pointers ψ from $\arg\max_j v_T(j)$.

CYK parsing. For a grammar in Chomsky normal form and words $w_1 \dots w_n$, cell $T[i, j]$ holds every non-terminal that derives $w_i \dots w_j$:

$$T[i,i] = \{A : A \to w_i\}, \qquad T[i,j] = \bigcup_{k=i}^{j-1} \{A : A \to B\,C,\ B \in T[i,k],\ C \in T[k{+}1, j]\}, \tag{5}$$

and the sentence is accepted when $S \in T[1, n]$. Each step of the explainer is one cell, with the split points tried.

Language-model fallback. With an API key, a problem that no solver recognises but that looks like a problem (it contains a number or a verb such as "solve" or "compute") may be explained by a language model. Such an explainer is labelled on the page as not computed by the system, has at most eight steps and is accepted only with at least three.

4 Interactive Labs

4.1 The lab bench

All labs share one page structure: presets, a view of the current state, a "what is happening" panel with *Step*, *Back* and *Play*, and a "try your own input" area. A lab supplies only its inputs and a function build$(x) \to \langle f_1, \dots, f_m \rangle$ that returns frames; each frame f_k is one small decision with its explanation and the view to show. This makes a new lab a single function, and it is also the contract we give to a language model when it writes a lab.

4.2 Choosing labs for a deck

Each built-in lab declares a list of topic phrases. A slide gets a built-in lab when its title or text matches one of them. Otherwise a lab may be generated, subject to a policy that keeps decks usable: one lab per topic (continuation slides share it), the same lab at most twice per deck, at most ten generated labs per deck, and no lab for agenda, summary or quiz slides or for titles that look like OCR debris (fewer than 70% letters, unbalanced brackets, or no word of four letters).

4.3 Generated labs

Model-written. With a key, the model receives the topic and the slide text and must return JSON with a title, input fields and JavaScript defining `PRESETS`,

Table 2. Held-out explainer benchmark

Measure	Result
Problems detected (initial run)	25 / 26
Problems detected (after one parsing fix)	26 / 26
Correct problem type	26 / 26
Correct answer	26 / 26
Every step has a reason	26 / 26
Mean steps per explainer	5.1
Median solve time	0.7 ms
False detections on lecture sentences	0 / 12

`fill`, `readInputs` and `build`. The code is rejected if it contains `fetch`, `XMLHttpRequest`, dynamic `import`, `eval`, cookies or local storage, or a closing script tag, and it must pass `node -check`. A lab never talks to the network.

Offline. Without a key, or when the model's lab is rejected, the lab is built from the visual structure the decision engine found on the slide: a process is walked step by step, a comparison is explored criterion by criterion, a timeline event by event, and a list of concepts is revealed one at a time after a "think first" prompt. A slide with fewer than three structured items gets no generated lab.

Labs are generated in parallel (four workers) before the deck is built and cached by a hash of their content, so building the same deck again costs nothing.

5 Evaluation

5.1 Explainer accuracy

We wrote 26 problems that do not appear in our unit tests—new numbers and new wordings across all solver types—and worked out each answer by hand. We also collected 12 ordinary sentences from our lecture notes (definitions, history, descriptions of algorithms) that must not be treated as problems. During this exercise two of our hand answers turned out to be wrong (a bigram product and a Viterbi path); the solvers were right in both cases.

The one initial miss was the phrasing "Find x if $4x+9 = x+21$"; the extractor did not remove the words before the equation. We added the pattern and two regression tests. We report both numbers because the fix was made after seeing the benchmark.

5.2 Labs on real decks

On three NLP lecture decks (36, 95 and 216 pages), built without an API key, the system attached labs to 6, 11 and 14 distinct topics. The 216-page syntax deck received four built-in labs (CYK parsing, Viterbi, bigram model, HMM tagging) and ten generated ones, which is the cap. With API keys, generating

labs sequentially had made a deck take 163 s; generating them in parallel before the build brought it to 74 s for the 36-page deck.

6 Discussion

Exact solvers cover a closed set of problem types, and this is a deliberate trade. Within that set the explanations are always correct and are produced in under a millisecond; outside it the system says honestly that the explanation came from a model, or shows nothing. The labs follow the same principle: a generated lab that cannot be checked is not shown.

Our evaluation measures correctness and coverage, not learning. The research on worked examples and on interactive visualisation [10, 4] suggests that the format can help, but whether these particular steppers and labs improve examination performance needs a classroom study, which we plan for the next semester.

7 Conclusion

Turning a solved problem on a slide into an explained, computed stepper, and a hard topic into a small lab, makes lecture decks something students can work through rather than only read. By computing every number and checking every generated program, the system can offer these features without the errors that make AI-generated study material hard to trust.

Acknowledgments. We thank our project mentor and the Department of Artificial Intelligence and Data Science, DJSCE, for guidance and for the lecture material used in this study.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Atkinson, R.K., Derry, S.J., Renkl, A., Wortham, D.: Learning from examples: Instructional principles from the worked examples research. Review of Educational Research **70**(2), 181–214 (2000)
2. Chi, M.T.H., Wylie, R.: The ICAP framework: Linking cognitive engagement to active learning outcomes. Educational Psychologist **49**(4), 219–243 (2014)
3. Freeman, S., Eddy, S.L., McDonough, M., Smith, M.K., Okoroafor, N., Jordt, H., Wenderoth, M.P.: Active learning increases student performance in science, engineering, and mathematics. Proceedings of the National Academy of Sciences **111**(23), 8410–8415 (2014)
4. Hundhausen, C.D., Douglas, S.A., Stasko, J.T.: A meta-study of algorithm visualization effectiveness. Journal of Visual Languages and Computing **13**(3), 259–290 (2002)

5. Ji, Z., Lee, N., Frieske, R., Yu, T., Su, D., Xu, Y., Ishii, E., Bang, Y.J., Madotto, A., Fung, P.: Survey of hallucination in natural language generation. ACM Computing Surveys **55**(12), 1–38 (2023)
6. Jurafsky, D., Martin, J.H.: Speech and Language Processing. Stanford University, 3rd (draft) edn. (2025), online manuscript, `https://web.stanford.edu/~jurafsky/slp3/`
7. Kasneci, E., et al.: ChatGPT for good? On opportunities and challenges of large language models for education. Learning and Individual Differences **103**, 102274 (2023)
8. Meurer, A., Smith, C.P., Paprocki, M., Čertík, O., et al.: SymPy: Symbolic computing in Python. PeerJ Computer Science **3**, e103 (2017)
9. Naps, T.L., Rößling, G., Almstrum, V., Dann, W., Fleischer, R., Hundhausen, C., Korhonen, A., Malmi, L., McNally, M., Rodger, S., Velázquez-Iturbide, J.Á.: Exploring the role of visualization and engagement in computer science education. ACM SIGCSE Bulletin **35**(2), 131–152 (2003)
10. Renkl, A.: Toward an instructionally oriented theory of example-based learning. Cognitive Science **38**(1), 1–37 (2014)
11. Roediger, H.L., Karpicke, J.D.: Test-enhanced learning: Taking memory tests improves long-term retention. Psychological Science **17**(3), 249–255 (2006)
12. Sweller, J., Cooper, G.A.: The use of worked examples as a substitute for problem solving in learning algebra. Cognition and Instruction **2**(1), 59–89 (1985)
13. VanLehn, K.: The relative effectiveness of human tutoring, intelligent tutoring systems, and other tutoring systems. Educational Psychologist **46**(4), 197–221 (2011)
14. Wieman, C.E., Adams, W.K., Perkins, K.K.: PhET: Simulations that enhance learning. Science **322**(5902), 682–683 (2008)
15. Wolfram Research: Wolfram|Alpha step-by-step solutions. `https://www.wolframalpha.com/pro/step-by-step-math-solver` (2026), accessed Oct. 2026